Documentación del proyecto de control de acceso por huella

Oct 9, 2026 · @saul

Resumen

El proyecto controla un acceso con huella digital: un sensor identifica a la persona, un Arduino mueve un servo y Node-RED registra el evento en MySQL.

Cada acceso reconocido queda guardado en la base de datos `dh`. Un dashboard muestra el último acceso y los últimos 10 registros.

Este documento reúne la arquitectura, el cableado, el código, la puesta en marcha y las limitaciones conocidas.

Objetivo y alcance

El objetivo es abrir un mecanismo solo cuando el sensor reconoce una huella registrada, y dejar constancia de cada acceso.

Incluye

- Alta de huellas en el sensor, con IDs del 1 al 127.
- Detección de huella y movimiento del servo.
- Registro de accesos en MySQL.
- Dashboard con el último acceso y el historial.

No incluye

- Alta de usuarios desde la web: se hace con una consulta SQL.
- Autenticación en el dashboard.
- Control de varias puertas o varios sensores.

Componentes y materiales

El sistema usa 6 elementos de hardware y 5 de software. Los modelos exactos del sensor y del servo son supuestos: confírmalos con tu material.

Tipo	Componente	Función
Hardware	Arduino Uno o Nano	Lee el sensor, mueve el servo y envía datos por USB. Se asume por el uso de SoftwareSerial en los pines 2 y 3.
Hardware	Sensor de huella óptico (R307 o Adafruit)	Guarda las huellas y compara el dedo.
Hardware	Servo (SG90 o similar)	Abre el mecanismo, conectado al pin 9.
Hardware	Protoboard y cables	Comparten 5V y GND entre componentes.
Hardware	Cable USB	Une el Arduino con la PC (puerto COM3).
Hardware	PC con Windows	Ejecuta Node-RED y MySQL.
Software	Arduino IDE con librerías `Adafruit_Fingerprint` y `Servo`	Programa la placa.
Software	Node-RED con Dashboard 2.0	Lógica del flujo y panel web.
Software	`node-red-node-serialport`	Lee el puerto serial.
Software	`node-red-node-mysql`	Consulta y guarda en la base.
Software	MySQL Community Server 8.0.17	Base de datos `dh`. Versión tomada del respaldo SQL.

Conexiones electrónicas

El sensor usa los pines 2 y 3 del Arduino y el servo usa el pin 9; ambos comparten 5V y GND.

Componente	Pin	Color del cable	Va a
Sensor	VCC	Rojo	Arduino 5V
Sensor	GND	Negro	Arduino GND
Sensor	TX	Amarillo (o verde)	Arduino D2
Sensor	RX	Blanco	Arduino D3

Componente	Pin	Color del cable	Va a
Servo	VCC	Rojo	Arduino 5V
Servo	GND	Marrón	Arduino GND
Servo	Señal	Naranja	Arduino D9
Arduino	USB	Cable USB	PC, puerto COM3

Al armarlo ten en cuenta lo siguiente:

- Los colores del sensor varían según el modelo. Guíate por las letras TX y RX del conector: TX del sensor va a D2 y RX del sensor va a D3.
- Si el Arduino se reinicia cuando el servo se mueve, alimenta el servo con una fuente externa de 5V y une su GND con el del Arduino.
- Si tu sensor es solo de 3.3V, conéctalo al pin 3.3V en lugar de 5V.
- La tabla también está en la hoja `conexiones_proyecto_huella.xlsx`.

Base de datos

La base `dh` tiene dos tablas y se crea con el archivo `detector_huella.sql`. Las dos tablas se entregan vacías.

Tabla	Columna	Tipo	Descripción
`usuarios_h`	`id_uh`	INT, clave primaria	ID de la huella guardada en el sensor
`usuarios_h`	`nombre_usuario`	VARCHAR(50)	Nombre que se muestra en el dashboard
`logs_acceso`	`usuario_id`	INT	ID de la huella que accedió
`logs_acceso`	`fecha`	DATETIME, por defecto la hora actual	Momento del acceso

El valor de `id_uh` debe ser igual al ID con el que se guardó la huella en el sensor. Si no coincide, el dashboard mostrará "Desconocido".

Para agregar un usuario de prueba:

```
INSERT INTO usuarios_h (id_uh, nombre_usuario) VALUES (1, 'Tu Nombre');
```

Firmware del Arduino

Hay dos sketches: uno para registrar huellas y otro para detectarlas y mover el servo. Los dos usan el monitor serial a 9600 baudios y el sensor a 57600.

Sketch	Qué hace
`guardar_huella.ino`	Pide un ID del 1 al 127, toma la huella dos veces y la guarda en el sensor.
`servo_detector_huella.ino`	Busca coincidencias cada 2 segundos. Si hay una, mueve el servo a 100 grados durante 5 segundos y lo regresa a 0.

El sketch de detección envía estas líneas por el puerto serial:

Mensaje	Significado
`Found a print match!`	El sensor reconoció la huella.
`Found ID #N with confidence of X`	ID reconocido y nivel de confianza. Node-RED lee esta línea.
`No match found`	La huella no está registrada. El servo no se mueve.

Mientras el servo está abierto, el código espera con `delay(5000)`, así que no lee otra huella durante esos 5 segundos.

Flujo de Node-RED

El flujo `proyecto_huella_dashboard2.json` reemplaza al script de Python original: lee el puerto serial, consulta MySQL y actualiza el dashboard.

Nodo	Tipo	Función
Arduino COM3	serial in	Recibe cada línea del Arduino.
Parsear Found ID	function	Extrae el ID de `Found ID #` con una expresión regular.
Buscar usuario	mysql	Consulta `usuarios_h` con el ID.
Evaluar acceso	function	Marca el acceso como concedido o denegado y guarda el último acceso.

Nodo	Tipo	Función
Guardar log	mysql	Inserta el registro en `logs_acceso`, solo si el usuario existe.
Preparar historial	function	Arma la consulta de los últimos 10 registros.
Últimos logs	mysql	Devuelve el historial.
Armar panel	function	Une el último acceso y el historial.
Panel huella	ui-template	Dibuja el panel en el dashboard.
Simular huella ID 1	inject	Prueba el flujo sin Arduino.
Al iniciar	inject	Carga el panel al desplegar el flujo.

Las consultas usan parámetros con nombre (`:id`) para evitar inyección SQL.

Configuración

- Puerto serial: `COM3` , 9600 baudios, separador de línea `\n` .
- MySQL: host `127.0.0.1` , puerto 3306, base `dh` . El usuario y la contraseña se escriben en el editor de Node-RED, no en el archivo.
- Panel: se asigna a un grupo del dashboard después de importar, por ejemplo `Group 1` .

Instalación y puesta en marcha

La puesta en marcha lleva 8 pasos, en este orden.

1. Crea la base de datos ejecutando `detector_huella.sql` en MySQL.
2. Carga `guardar_huella.ino` en el Arduino y registra cada huella con un ID del 1 al 127.
3. Carga `servo_detector_huella.ino` en el Arduino.
4. Agrega cada usuario en `usuarios_h` con el mismo ID que usaste en el sensor.
5. En Node-RED, instala `node-red-node-serialport` y `node-red-node-mysql` desde Manage palette.
6. Importa `proyecto_huella_dashboard2.json` desde Menú, Import.
7. Asigna el grupo al nodo Panel huella, escribe el usuario y la contraseña de MySQL, revisa que el puerto sea `COM3` y pulsa Deploy.
8. Abre el dashboard con Open Dashboard y prueba con el nodo Simular huella ID 1.

Cierra el Monitor Serie de Arduino y el script de Python antes de usar Node-RED: el puerto COM3 solo lo puede abrir un programa a la vez.

Si MySQL 8 devuelve el error `ER_NOT_SUPPORTED_AUTH_MODE`, cambia el método de autenticación del usuario:

```
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY
'tu_clave';
```

Pruebas

Son 5 casos de prueba. Los resultados son los esperados según el código; hay que confirmarlos en el equipo real.

Nº	Prueba	Resultado esperado
1	Pulsar Simular huella ID 1 con el usuario 1 en `usuarios_h`	El panel muestra su nombre con borde verde y aparece una fila nueva en el historial.
2	Pulsar Simular huella ID 1 sin ese usuario en la base	El panel muestra "Desconocido" con borde rojo y no se guarda ningún registro.
3	Poner una huella registrada en el sensor	El servo gira 100 grados durante 5 segundos y el panel se actualiza.
4	Poner una huella no registrada	El Arduino envía `No match found`, el servo no se mueve y el panel no cambia.
5	Abrir el Monitor Serie de Arduino mientras Node-RED corre	Node-RED no puede abrir `COM3`; hay que cerrar el Monitor Serie.

Comportamiento ante fiallas

El control de acceso no depende de la red: el sensor decide y el Arduino mueve el servo por su cuenta. Lo que sigue es el comportamiento esperado según el código, sin probar en hardware. Los archivos del proyecto no incluyen ninguna Raspberry Pi; todo corre en una PC.

Falla	Qué pasa	Qué falta resolver
El Arduino pierde energía	Dejan de llegar datos. Las huellas siguen en el sensor y los registros siguen en MySQL. Al volver la energía, el servo regresa a 0.	El dashboard sigue mostrando el último acceso sin avisar que el Arduino está fuera de línea.

Falla	Qué pasa	Qué falta resolver
Se corta la energía con el servo abierto	El servo queda donde lo deje la mecánica.	El código no define un estado seguro.
Se cae la red	Si Node-RED y MySQL están en la misma PC, solo se pierde el acceso remoto al dashboard.	Nada, en la configuración actual.
MySQL no responde	Las consultas fallan y los accesos de ese periodo no se guardan.	No hay cola ni reintento.
El sensor reconoce a alguien que no está en `usuarios_h`	El servo abre igualmente; el panel marca "Desconocido".	Decidir si el servo debe depender de la base de datos.

Seguridad y limitaciones

El riesgo más urgente es la contraseña de MySQL, que está escrita en texto plano en `python_coneccion.py`.

- Cambia esa contraseña si compartes o subes el archivo a algún sitio.
- El dashboard no tiene autenticación: cualquiera con acceso a la red del equipo puede verlo.
- El servo abre por decisión del sensor, aunque la base de datos no reconozca al usuario.
- Si MySQL falla, los accesos de ese momento se pierden, porque no hay búfer.
- No hay forma de saber que el Arduino se desconectó hasta que alguien lo nota.
- Las huellas viven solo en el sensor y el sketch de alta admite IDs del 1 al 127.
- La hora del último acceso sale del reloj de la PC que ejecuta Node-RED.

Mejoras fiuturas

Las dos primeras cierran las fallas más importantes de la sección anterior.

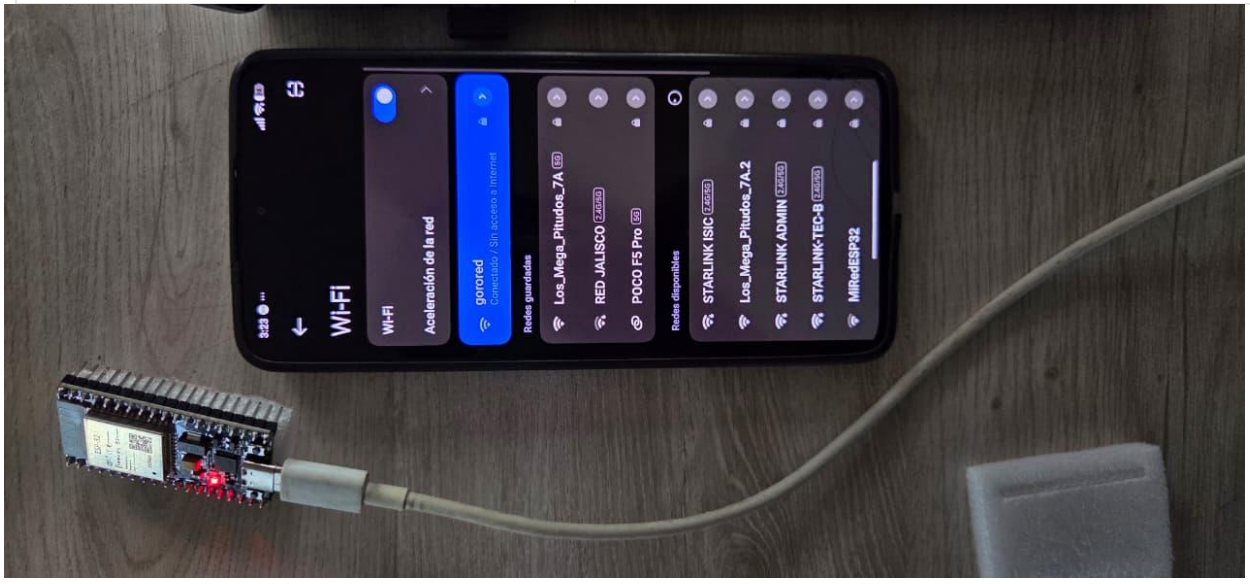
- [ ] Enviar un latido desde el Arduino y mostrar "Arduino sin comunicación" en el dashboard cuando deje de llegar.
- [ ] Guardar los accesos en un archivo cuando MySQL falle y reintentar al volver, con un nodo `catch`.
- [ ] Hacer que el servo abra solo si el usuario existe en la base de datos.

- [ ] Reemplazar `delay(5000)` por un temporizador con `millis()` para seguir leyendo mientras el servo está abierto.
- [ ] Agregar un formulario para dar de alta usuarios desde el dashboard.
- [ ] Proteger el dashboard con usuario y contraseña.

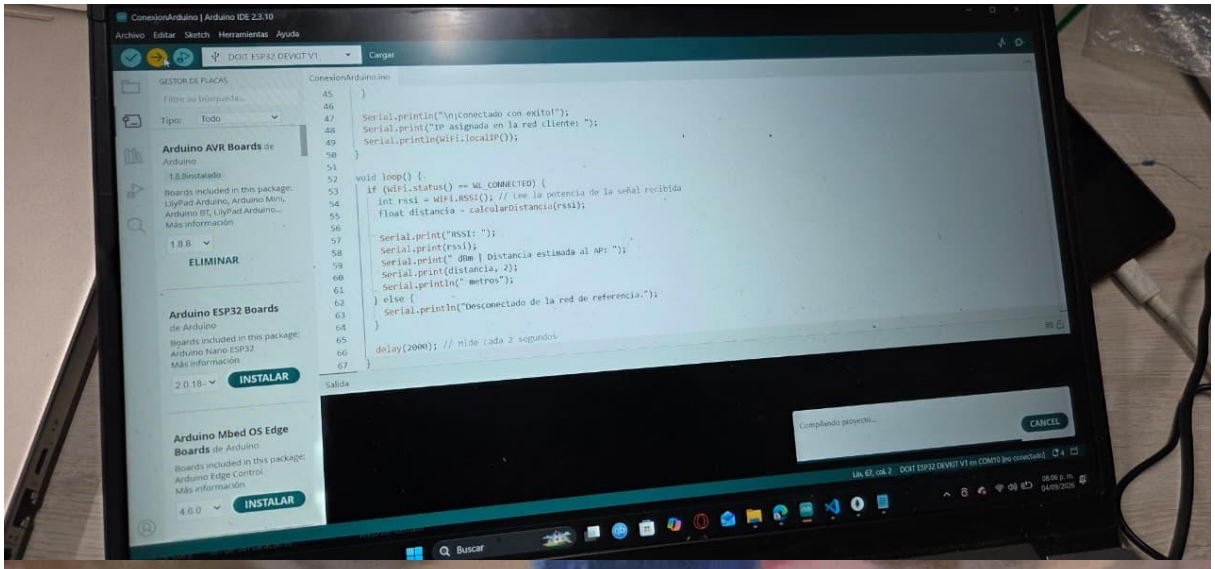
Anexo: archivos del proyecto

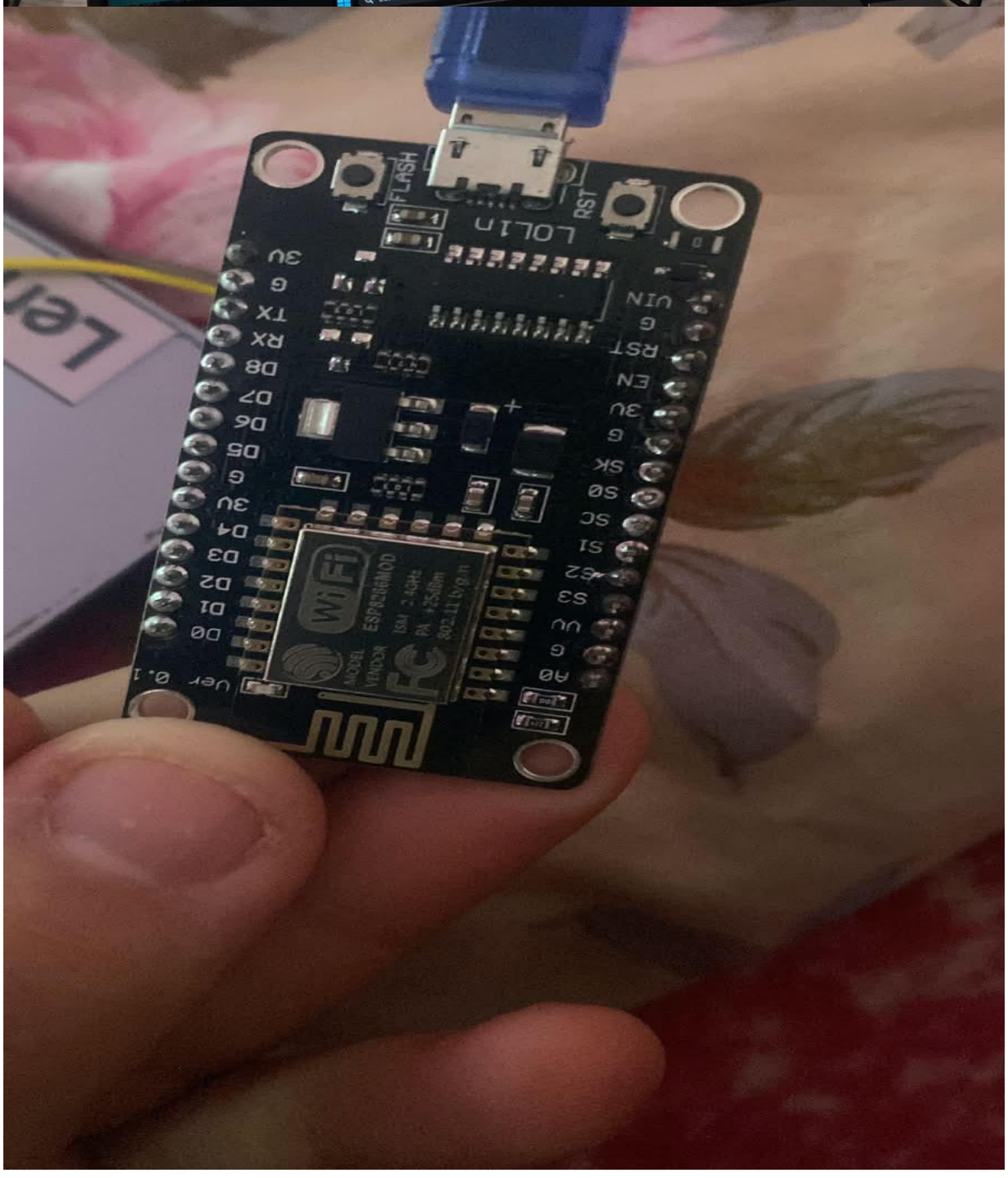
El proyecto se compone de 7 archivos.

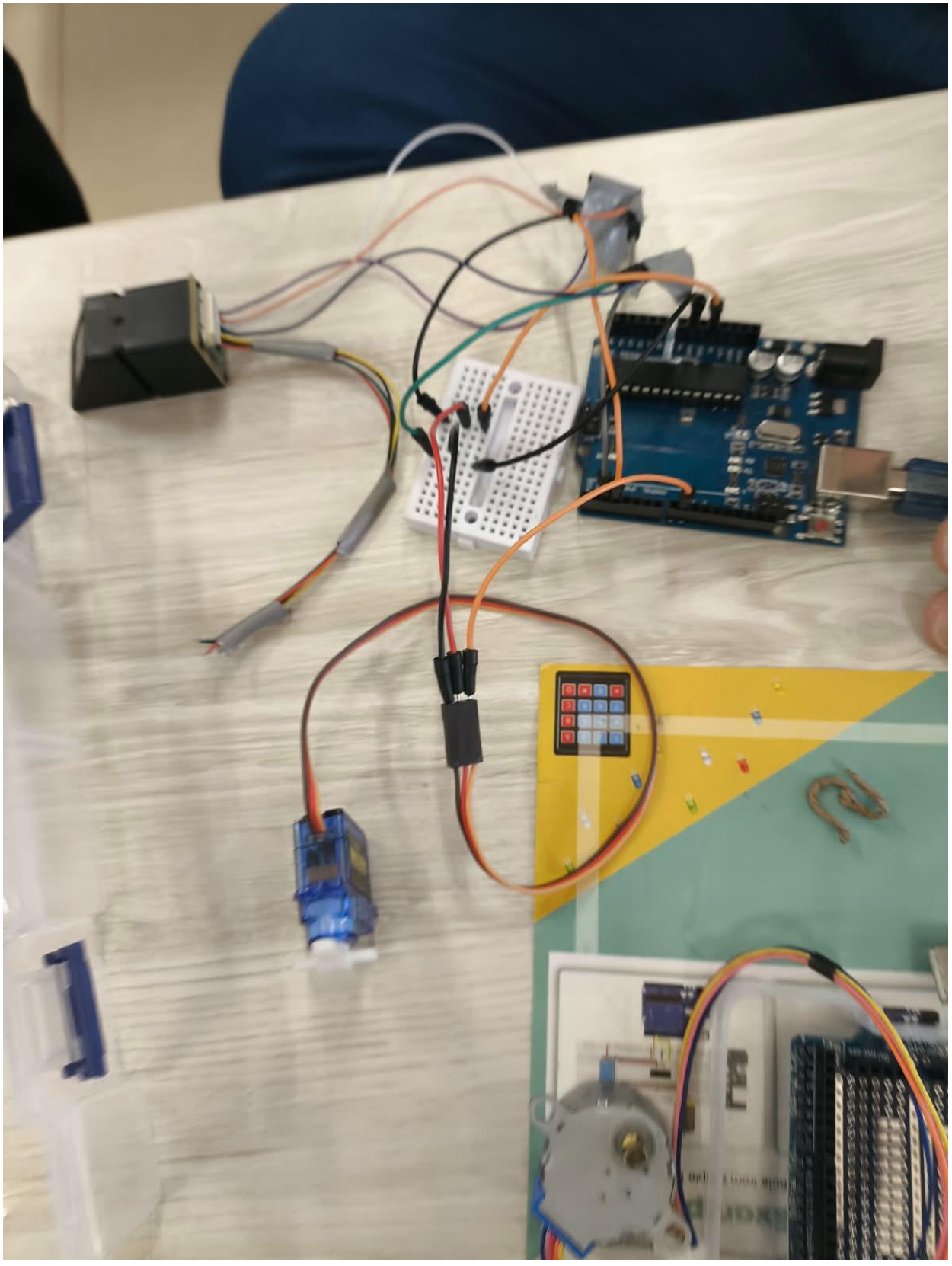
Archivo	Descripción
`detector_huella.sql`	Crea la base `dh` y las tablas `usuarios_h` y `logs_acceso`.
`guardar_huella.ino`	Sketch de Arduino para registrar huellas en el sensor.
`servo_detector_huella.ino`	Sketch de Arduino para detectar huellas y mover el servo.
`python_coneccion.py`	Versión original con Flask, `pyserial` y `mysql.connector`. La reemplaza el flujo de Node-RED.
`proyecto_huella_dashboard2.json`	Flujo de Node-RED para Dashboard 2.0.
`proyecto_huella_flow.json`	Primera versión del flujo, con una página web en `/huella` en lugar del dashboard.
`conexiones_proyecto_huella.xlsx`	Hoja de cálculo con la tabla de cableado y las notas de armado.

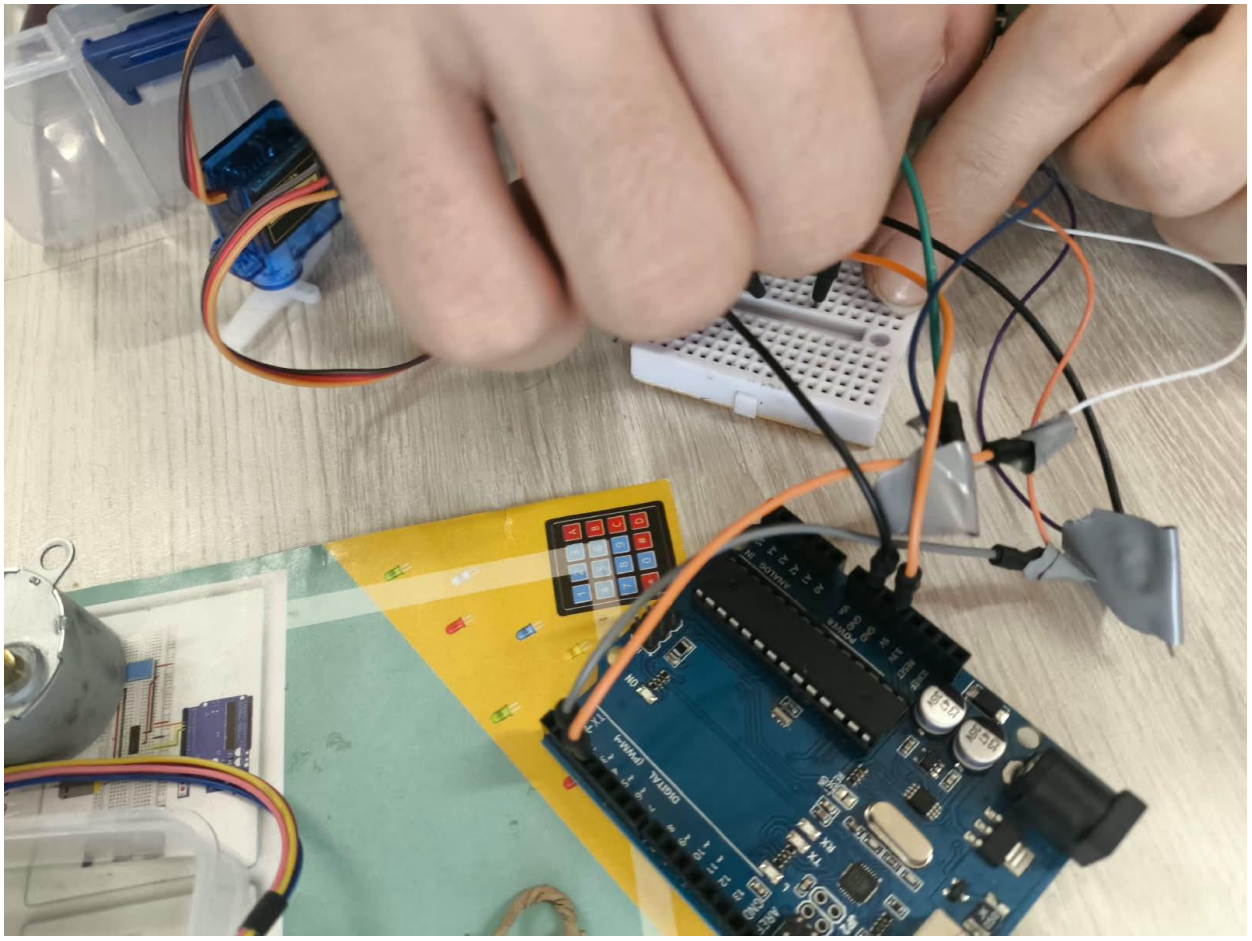





ConexionArduino | Arduino IDE 2.3.10
Archivo Editar Sketch Herramientas Ayuda
DOIT ESP32 DEVKIT V1
Cargar
GESTOR DE PLACAS
ConexionArduino.ino
Arduino AVR Boards de Arduino
ELIMINAR
Arduino ESP32 Boards de Arduino
INSTALAR
Arduino Mbed OS Edge Boards de Arduino
INSTALAR
Serial.println("\n¡Conectado con éxito!");
Serial.print("IP asignada en la red cliente: ");
Serial.println(WiFi.localIP());
void loop() {
if (WiFi.status() == WL_CONNECTED) {
int rssi = WiFi.RSSI(); // Lee la potencia de la señal recibida
float distancia = calcularDistancia(rssi);
Serial.print("RSSI: ");
Serial.print(rssi);
Serial.print(" dBm | Distancia estimada al AP: ");
Serial.print(distancia, 2);
Serial.println(" metros");
} else {
Serial.println("Desconectado de la red de referencia.");
}
delay(2000); // Mide cada 2 segundos
Salida
Compilando proyecto...
CANCEL
Buscar


FLASH
RST
LOLIN
3V
G
TX
RX
D8
D7
D6
D5
G
3V
D4
D3
D2
D1
D0
A0
VIN
G
RST
EN
3V
G
SK
SO
SC
SI
SD2
SD3
5V
G
WiFi
ESP8266MOD




DIGITAL (PWM~)
POWER
ANALOG IN

